

使用多重軌跡搜尋演算法解決二次分配問題

曾伶玉^{1,2} 吳志賢¹

¹國立中興大學資訊科學與工程學系

²國立中興大學資訊網路與多媒體研究所

lytseng@cs.nchu.edu.tw, s9756003@cs.nchu.edu.tw

摘要

二次分配問題是一個 NP-hard 的問題，它屬於組合最佳化的問題。現實生活中有很多問題可以化成二次分配問題。例如：打字機鍵盤設計問題、學校及醫院配線設計問題及排班問題等。

本研究中提出一新的架構並結合基因演算法及區域搜尋演算法來解二次分配問題。針對目前為止所發展的演算法的弱點，我們提出下列方法加以克服：1. 利用新的架構對解空間做更廣泛且有效率的搜尋；2. 提出在解空間對搜尋範圍作縮放之機制加強對搜尋之引導；3. 搭配多種區域搜尋演算法以達到更細部的搜尋。

我們應用本論文所提出之演算法作測試在 Tseng 與 Liang 在 2006 年及 James Rego 與 Glover 在 2009 年所匯整的四個題組中，並和其他演算法進行比較，獲得了不錯的成果。

一. 簡介

二次分配問題 [1] (quadratic assignment problem, QAP) 為一 NP-hard 的組合最佳化問題，最早在 1957 年由 Koopmans 與 Beckmann 所提出，其目的是要將設備分配到適當的位置使得整體的總成本為最小。二次分配問題被廣泛應用到各個領域，例如：電子領域的麵包板配線問題、化學反應的分析、工廠內機器排班問題、統計資料分析。另外，一些知名的問題也可以簡化成二次分配問題。例如：旅行銷售員問題(traveling salesman problem, TSP)、最大完全子圖問題(maximum clique problem)、線性排列問題(the linear ordering problem) 和 圖形著色問題 (graph-coloring problem)。

雖然二次分配問題早在 1957 年就已經被提出，且在 1976 年，Sahni 和 Gonzales [17] 證明 QAP 是 NP-hard 的問題。由於二次分配問題的解空間複雜度為 $O(n!)$ ，所以早期的文獻所提出的方法只適用在問題變數不多之情況下，因為面對問題變數較多時，將會花太多時間，造成搜尋上的困難，

因此超啟發演算法(metaheuristic algorithm)紛紛被提出，此類演算法由一組或多組任意解出發並利用不同的架構和搜尋方式在解空間(solution space)中搜尋更好的解。

超啟發演算法應用在二次分配問題，效果非常顯著，例如：包括類神經網路演算法(artificial neural networks, ANN) [2]、模擬退火演算法(simulated annealing, SA) [4] [22]、基因演算法(genetic algorithm, GA) [20] [14]、禁忌搜尋演算法(tabu search, TS) [19], [15], [11]、演化式策略演算法(evolution strategies) [16]、蟻群最佳化演算法(ant colony optimization, ACO) [10], [13]。

近幾年，有學者提出結合上述方法的演算法，其效果也很顯著，例如：結合蟻群最佳化演算法和基因演算法[21]、結合蟻群最佳化演算法和模擬退火演算法[5]、結合基因演算法和禁忌搜尋演算法[7], [8], [6]、Tabu search 結合 multistart [9] 及 Stutzle [18] 提出 4 個版本的區域搜尋演算法(ILS1~ILS4)。其中 DivTS [12] 和 ANGEL 演算法[21] 是目前為止做得較好的論文，我們將在實驗中和這兩篇論文做一比較。

二. QAP 問題定義

n 個設備(facilities)， n 個位置(locations)，每個設備之間有流量(flow)，每個位置之間有距離(distance)，如何將這 n 個設備分配到 n 個位置使得整體的總成本(total cost)為最小。成本計算公式：

$$\sum_{i=1}^n \sum_{j=1}^n f_{ij} d_{x(i)x(j)}$$

f_{ij} 為設備 i 和設備 j 之間的流量， d_{ij} 為設備 i 和設備 j 之間的距離， x 為一組 $1, \dots, n$ 所排列的解， $x(i)$ 則代表設備 i 所對應的位置。以圖 1 為例：紅色圓圈代表設備，藍色方形代表位置，則 $x = \{4, 2, 3, 1\}$ 則代表設備 4 被分配在第 1 的位置，設備 2 被分配在第 2 的位置，設備 3 被分配在第 3 的位置，設備 1 被分配在第 4 的位置，根據解 x 即可算出成本。

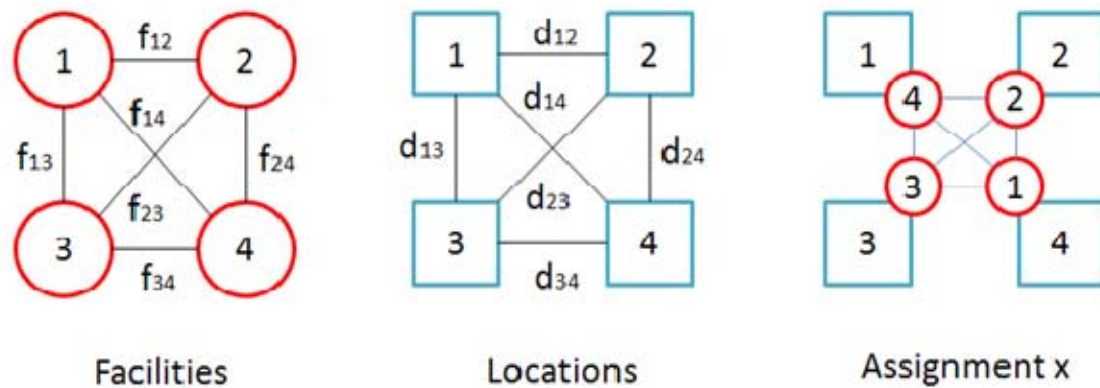


圖 1. 二次分配問題定義

三. 多重軌跡搜尋演算法

在本章中，我們將介紹本論文所提出的多重軌跡搜尋(Multiple Trajectory Search)演算法，以下簡稱 MTS。並依序介紹 MTS 之架構、概念、特性和 MTS 底層所搭配的中範圍區域搜尋(Right Search)演算法。

3.1 MTS 主要架構

本論文利用多重軌跡搜尋(MTS)演算法對解空間的各區域作有系統、且具效率的搜尋。以下為 MTS 整體架構和流程圖(圖 2)：

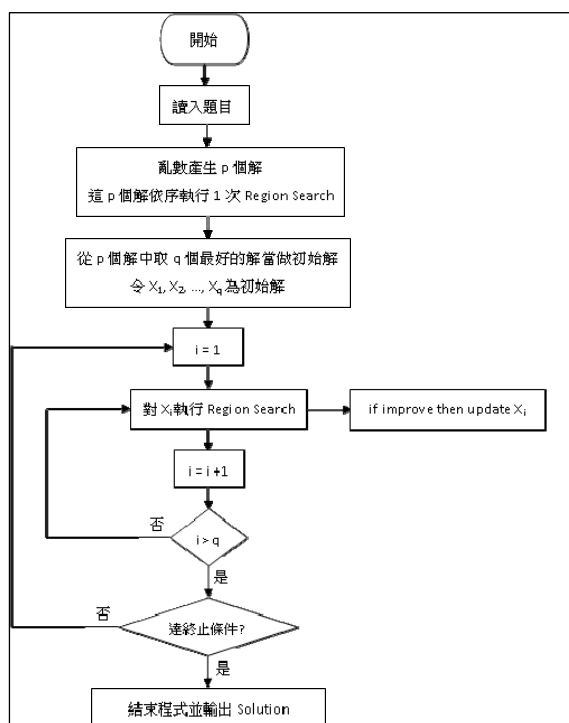


圖 2. 多重軌跡搜尋演算法流程圖

1. 初始化：讀取題目並以亂數產生 p 個解： $X_1, X_2, \dots, X_p$ ，對這 p 個解依序執行一次中範圍區域搜尋(Right Search)，執行完 Region Search 後從 p 個解中取 q 個最好的解當做初始解： $X_1, X_2, \dots, X_q$ 。
2. 對所有解 X_i 執行中範圍區域搜尋(Right Search)演算法，如果 X_i 有進步，則更新 X_i 。然而 Region Search 包含了產生初始解、基因演算法(genetic algorithm)、區域搜尋(local search)，這些運算將會在下面小節中做詳細介紹。
3. 反覆執行步驟 2 直到找到最佳解(optimal solution)或達到終止條件為止。

3.2 MTS 之概念、特性

MTS 演算法的搜尋過程非常有系統，它可以避免重複在某一區域做無謂的搜尋，而是有系統的在解空間(Solution Space)之各個區域做搜尋，如果最佳解(Optimal Solution)剛好落在搜尋的區域附近，則相當有機會透過 MTS 底層的中範圍區域搜尋(Right Search)演算法作更詳細的搜尋以找到最佳解。

然而一個好的演算法必須將廣度搜尋能力(Diversification)和深度搜尋能力(Intensification)這兩股搜尋力道拿捏得恰到好處才可以找到品質優良的解。本篇論文所提出的 MTS 上層負責的是廣度搜尋，深度搜尋主要是靠 MTS 底層所搭配的中範圍區域搜尋演算法來達成，中範圍區域搜尋演算法可以引導搜尋方向，使搜尋的方向可以往較好的解邁進，圖 3 與圖 4 表示 MTS 的搜尋概念。

此外，MTS 演算法的優點是不拘泥於問題的特性上，因此並不受到問題本身的限制，如果要解其他的問題時，只需要抽換掉底層的 Region Search 演算法，改為適合該問題的 Region Search 演算法即可。如此可以節省程式開發時間，並把時間花在研究適合問題本身的 Region Search 演算法即可。

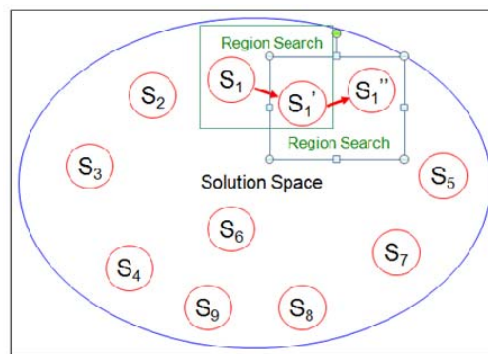


圖 3. MTS 概念圖(有找到更好的解，則移過去)

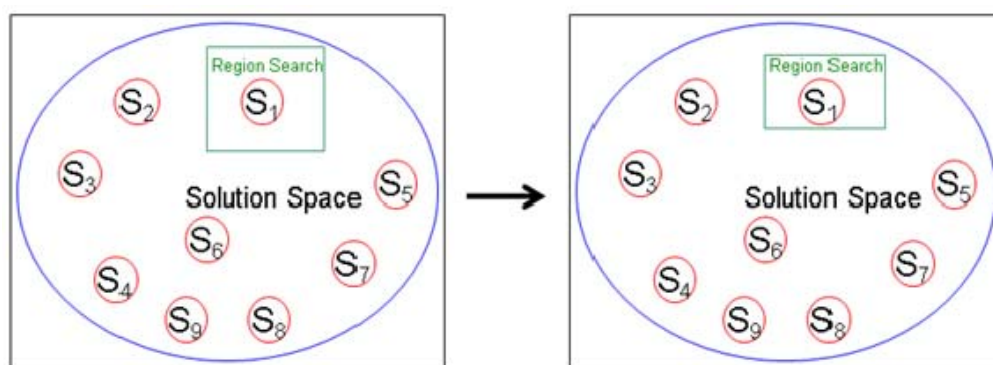


圖 4. MTS 概念圖(沒有找到更好的解，則縮小搜尋範圍)

3.3 中範圍區域搜尋演算法

3.3.1 中範圍區域搜尋主要架構

中範圍區域搜尋是由灑點(產生初始解)、基因演算法和區域搜尋(Local Search)演算法所組成，主要功能是負責底層較細部的搜尋，也就是前面所描述的深度搜尋能力(Intensification)。以下為 Region Search 整體架構和流程：

1. 依據 X_i solution 和 sizeRegionSearch 執行灑點並產生 GLS 的 initial population。
2. 從 GLS Population 隨機挑選 2 個父代。
3. 使用 Crossover 產生 4 個子代，取最好的兩個當子代並執行 Local Search。
4. 重複執行步驟 2 與 3，直到滿足 Crossover 次數為止。
5. 從 GLS Population 取三個最好的 Solutions 執行 Local Search。
6. 如果此代沒有找到更好的解，則 $\text{count} = \text{count} + 1$ 。
7. 如果此代有找到更好的解，則 count 會重

設為 0。

8. 如果 count 等於 4，則 $\text{sizeRegionSearch} = \text{sizeRegionSearch} - 1$ ，否則返回步驟 2。
9. 如果 sizeRegionSearch 小於 4，則結束此程序，否則返回步驟 1。

3.3.2 灑點

在中範圍區域搜尋中，我們利用基因區域搜尋演算法(GLS)來作搜尋，在基因演算法裡的初始族群(Initial Population)，對所給的一個解，我們採用隨機抽取 k 個元素，再隨機放回去，以產生基因演算法的初始族群，然後再對這些初始族群做搜尋的動作。然而為了讓搜尋更有效率，我們在搜尋過程中根據搜尋情況逐步地讓搜尋範圍縮小，也就是抽取較少的元素，然後再隨機地放回。雖然看似簡單的動作，但對搜尋過程非常有幫助。我們可以在圖 5 中舉一個詳細的例子。

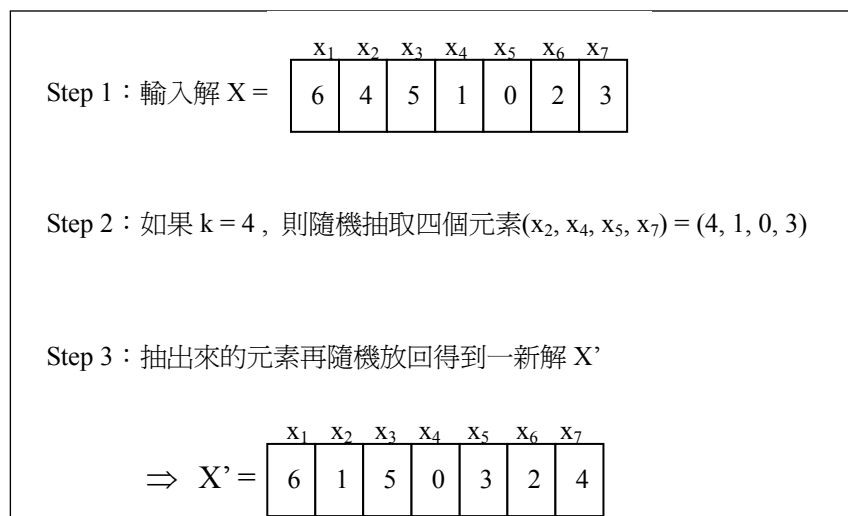


圖 5. 灑點演算法例子

3.3.3 基因演算法(Genetic algorithm)

基因演算法的組成要素主要包含產生初始族群 (Initial Population)、選擇 (Selection)、交配 (Crossover)、突變 (Mutation)，但本篇論文所採用的基因演算法將不使用突變運算。以下將介紹選擇、交配等基因演算法的運算。

1. 選擇 (Selection) : 從兩個父代與作區域搜尋的兩個子代中選出最好的兩個。
2. 交配 (Crossover) : 本論文採用 Tseng 與 Liang [21] 於 2006 年所提出的 Crossover OX1，以

下將描述 Crossover OX1 運算，並以圖 6 說明之。首先，將步驟一所選擇的 2 個父代分別命名為 Π_{p1} 、 Π_{p2} ，而父代交配所產生的子代分別命名為 Π_{c1} 、 Π_{c2} 。再來，隨機選取兩點 a 、 b ，將 Π_{p1} 、 Π_{p2} 兩個父代 a 點和 b 點間的 elements 分別複製到 Π_{c1} 、 Π_{c2} 的相對應的位置，最後 Π_{c1} 其餘的 elements 則依據 Π_{p2} 從 b 點以後的 elements 排列順序填滿為止； Π_{c2} 同理。

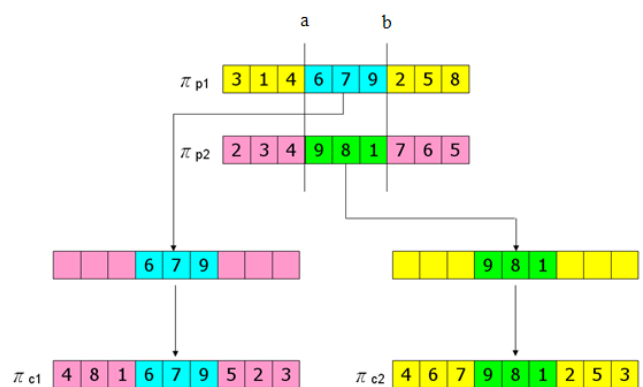


圖 6. 交配(Crossover)示意圖

3.3.4 區域搜尋演算法(Local Search)

在解空間裡的每一個點 (Solution) 都會有其鄰域 (neighborhood)，區域搜尋 (Local Search) 的目的是負責深度搜尋，以一個點 (初始解) 為出發點，並在其周遭的鄰域空間中搜尋更好的解，若有發現較好的解，隨即更新目前的解，這意味著初始解

將往較好或是較有可能找到最佳解的區域移動過去，再以更新過後的解為出發點，在其周遭的區域再次灑點來做搜尋。區域搜尋的範圍相較於整體的解空間其範圍較小，所以在 MTS 之中，區域搜尋 (Local Search) 負責細部的搜尋，仔細搜尋 MTS 每一個階段產生的解附近之鄰域空間，帶領搜尋方向逐步往較佳的區域邁進，在解空間的觀

點上，每次更新解都可抽象的當作點在移動，點的移動漸漸的形成了軌跡，並在軌跡周圍的 neighborhood 做仔細地搜尋且可以控制搜尋的範圍，並非盲目地隨意搜尋。而我們的區域搜尋演算法有兩種，以下將詳細敘述其作法。

1. Local Search 1 演算法：

Step1：列出計算 cost 之過程中的每一項(流量*距離)，如圖 7。

Step2：找出流量乘以距離為最大的數出來，如圖 7 最大的數為 30*60。並檢視由哪兩個設備所構成，將兩個設備命名為 A 和 B。

Step3：隨機選取一個設備，假設選到 A，則把距離 A 最近的一些設備列出來，假設這些是{C, D, E, F, G}，則依序將 A 和 C、D、E、F、G 交換，如果解有進步，就交換這兩個設備。

a=flow=	0	5	10	15
	5	0	20	25
	10	20	0	30
	15	25	30	0

b=distance=	0	10	20	30
	10	0	40	50
	20	40	0	60
	30	50	60	0

$X=(1,2,3,4)$

$$\begin{aligned} \text{Cost} &= 0*0 + 5*10 + 10*20 + 15*30 + \\ & 5*10 + 0*0 + 20*40 + 25*50 + \\ & 10*20 + 20*40 + 0*0 + 30*60 + \\ & 15*30 + 25*50 + 30*60 + 0*0 \\ &= 9100 \end{aligned}$$

圖 7. Local Search 1 例子

例子：圖 7 中 QAP 的成本是由一些流量乘以距離的和所組成，我們可以發現最大項是 30*60，然而此項主要是設備 3 和設備 4 的流量和距離所構成，所以我們可以将兩個設備其中一個，例如選設備 4 跟離它最近的設備 1 做交換，交換之後成本由原本的 9100 降為 5900，如圖 8 所示。

a=flow=	0	5	10	15
	5	0	20	25
	10	20	0	30
	15	25	30	0

b=distance=	0	10	20	30
	10	0	40	50
	20	40	0	60
	30	50	60	0

$X'=(4,2,3,1)$

$$\begin{aligned} \text{Cost} &= 0*0 + 5*50 + 10*60 + 15*30 + \\ & 5*50 + 0*0 + 20*40 + 25*10 + \\ & 10*60 + 20*40 + 0*0 + 30*20 + \\ & 15*30 + 25*10 + 30*20 + 0*0 \\ &= 5900 \end{aligned}$$

圖 8. Local Search 1 例子

2. Local Search 2 演算法：

Step1：建立表格，此表格紀錄兩兩設備交換所增加或減少的成本量。

Step2：找出負數值所對應的兩個設備做交換。如果沒有負數值，則不執行交換。

由於此演算法需要計算兩兩設備交換所增加或減少的成本量，計算複雜度為 $O(n^4)$ ，非常耗費計算時間，所以我們使用 Tseng 與 Liang [21]於 2006 年所提出的快速算法公式，然而此公式只能

用於特定的題目，也就是題目的流量矩陣和距離矩陣必須是對稱矩陣且對角線元素必須是 0 的情況下才可以使用。公式如下所示：

$$\Delta(x, r, s) = 2 \sum_{k \neq r, s}^n (f_{sk} - f_{rk})(d_{x(r)x(k)} - d_{x(s)x(k)})$$

x 為一 Solution， r 、 s 為欲交換的兩個設備， f 和 d 分別為兩設備間的流量和距離。使用快速算法公式，此演算法的計算複雜度由原本的 $O(n^4)$ 降為 $O(n^3)$ ，雖然計算複雜度有降低，但如果大量使用此演算法，則需耗費相當多的計算時間，所以為了節省計算時間，此演算法只有在特殊的情況下才會使用。以下將舉一個例子說明 Local Search 2 演算法的運作方式。

例子：

flow=	0	5	10	15
	5	0	20	25
	10	20	0	30
	15	25	30	0

distance=	0	10	20	30
	10	0	40	50
	20	40	0	60
	30	50	60	0

圖 9. Local Search 2 例子

$X = (1, 2, 3, 4) \Rightarrow \text{cost} = 9100$

$$\Delta(X, 1, 2) = 2 * [(20 - 10)(20 - 40) + (25 - 15)(30 - 50)] = -800$$

$$\Delta(X, 1, 3) = 2 * [(20 - 5)(10 - 40) + (30 - 15)(30 - 60)] = -1800$$

$$\Delta(X, 1, 4) = 2 * [(25 - 5)(10 - 50) + (30 - 10)(20 - 60)] = -3200$$

$$\Delta(X, 2, 3) = 2 * [(10 - 5)(10 - 20) + (30 - 25)(50 - 60)] = -200$$

$$\Delta(X, 2, 4) = 2 * [(15 - 5)(10 - 30) + (30 - 20)(40 - 60)] = -800$$

$$\Delta(X, 3, 4) = 2 * [(15 - 10)(20 - 30) + (25 - 20)(40 - 50)] = -200$$

$$\text{則 } X' = (4, 2, 3, 1) \text{ 的 } \text{cost} = X \text{ 的 } \text{cost} + \Delta(X, 1, 4) = 9100 + (-3200) = 5900$$

下列矩陣則表示兩兩設備交換所增加或減少的成本量：

0	-800	-1800	-3200
-800	0	-200	-800
-1800	-200	0	-200
-3200	-800	-200	0

矩陣中的第 i 列第 j 行的數字表示設備 i 和設備 j 交換後所增加或減少的成本量。也就是如果設備 1 和設備 2 交換，則整體成本會減少 800；同理，設備 1 和設備 4 交換，則整體成本會減少 3200。

四. 實驗結果

目前實驗的結果，是依照本論文所提出的多軌跡搜尋演算法來解二次分配問題，並且與近年來

所提出的超啟發式演算法作比較，使用 C++ 程式語言，測試平台為個人電腦使用 Intel Dual Core 2.33GHz CPU、3.5GB RAM、作業系統為 Windows XP，並對 QAPLIB [3] 所提供的 126 個題目做測試，實驗執行過程均使用單一核心，無平行處理運算，並將結果紀錄在下表格 1 至表格 4 中，表格 5 則是與其他演算法之效能比較。

表格 1 至表格 4 中每一欄位由左至右為：題目名稱、已知最佳解、解到最佳解的次數、平均的解和已知最佳解的差距(deviation, %)、最好的解和已知最佳解的差距(deviation, %)、執行時間(分鐘)。

Tseng 與 Liang 的 ANGEL [21] 演算法使用 Java 程式語言、測試平台為 Intel Pentium 2.0GHz CPU (PC)、1GB RAM、作業系統為 windows XP。James、Rego 與 Glover 的 DivTS [12] 演算法使用 C 程式語言、測試平台為 Intel Itanium 1.3GHz CPU (Server)、作業系統為 Linux SGI Altix。

4.1 實驗題組介紹

實驗題目共有 126 題。這 126 題公布在 QAPLIB [22] (<http://www.seas.upenn.edu/qaplib/inst.html>)。為了方便比較，我們採用 Stutzle [20] 的分類標準。這個分類標準會產生四個類型的題組：

1. Unstructured, Randomly Generated Instances：這類型题目的距離矩陣是依據均勻分配隨機產生的。包含的題目有：tai20a~tai100a、rou12~rou20、lipa20a~lipa90a、lipa20b~lipa90b，共有 28 題。
2. Instances with Grid – Distances：這類型题目的距離矩陣是依據兩點間的 Manhattan distance 所產生的。包含的題目有：nug12~nug30、sko42~sko100f、scr12~scr20、tho30~tho150、wil50~wil100，共有 36 題。
3. Real-Life Instances：這類型题目是依據現實生活中的應用所產生的。包含的題目有：bur26a~bur26h、els19、had12~had20、chr12a~chr25a、kra30a~kra32、ste36a~ste36c、esc16a~esc128，共 53 題。
4. Real-Life Like Instances：這類型题目是依據類似現實生活中的應用所產生的。包含的題目有：tai20b~tai100b，共 9 題。

4.2 MTS 實驗結果

表格 1. MTS 於題組一之實驗結果(共 28 題)

Type 1 - Unstructured, Randomly Generated					
Problem	BKS	Times	APD	BPD	Time(min.)
tai20a	703482	10	0.000	0.000	0.026
tai25a	1167256	8	0.092	0.000	1.354
tai30a	1818146	5	0.145	0.000	3.256
tai35a	2422002	2	0.357	0.000	4.259
tai40a	3139370	0	0.457	0.269	8.359
tai50a	4938796	0	0.687	0.358	19.562
tai60a	7205962	0	0.786	0.306	42.633
tai80a	13515450	0	0.756	0.569	76.596
tai100a	21059006	0	0.963	0.803	126.531
rou12	235528	10	0.000	0.000	0.011
rou15	354210	10	0.000	0.000	0.027
rou20	725522	10	0.000	0.000	0.046
lipa20a	3683	10	0.000	0.000	0.096
lipa20b	27076	10	0.000	0.000	0.021
lipa30a	13178	10	0.000	0.000	0.214
lipa30b	151426	10	0.000	0.000	0.052
lipa40a	31538	10	0.000	0.000	0.685
lipa40b	476581	10	0.000	0.000	0.064
lipa50a	62093	10	0.000	0.000	2.252
lipa50b	1210244	10	0.000	0.000	0.088
lipa60a	107218	8	0.001	0.000	0.201
lipa60b	2520135	10	0.000	0.000	0.101
lipa70a	169755	6	0.070	0.000	54.356
lipa70b	4603200	10	0.000	0.000	0.169
lipa80a	253195	4	0.589	0.000	76.364
lipa80b	7763962	10	0.000	0.000	0.265
lipa90a	360630	3	0.416	0.000	138.368
lipa90b	12490441	10	0.000	0.000	0.365

表格 2. MTS 於題組二之實驗結果(共 36 題)

Type 2 - Instances with Grid-Distances					
Problem	BKS	Times	APD	BPD	Time(min.)
nug12	578	10	0.000	0.000	0.018
nug14	1014	10	0.000	0.000	0.020
nug15	1150	10	0.000	0.000	0.052
nug16a	1610	10	0.000	0.000	0.030
nug16b	1240	10	0.000	0.000	0.030
nug17	1732	10	0.000	0.000	0.049
nug18	1930	10	0.000	0.000	0.052
nug20	2570	10	0.000	0.000	0.058
nug21	2438	10	0.000	0.000	0.062
nug22	3596	10	0.000	0.000	0.078
nug24	3488	10	0.000	0.000	0.096
nug25	3744	10	0.000	0.000	0.107
nug27	5234	10	0.000	0.000	0.121
nug28	5166	10	0.000	0.000	0.128
nug30	6124	10	0.000	0.000	0.130
sko42	15812	10	0.000	0.000	0.460
sko49	23386	6	0.022	0.000	3.840
sko56	34458	8	0.003	0.000	5.840
sko64	48498	10	0.000	0.000	3.840
sko72	66256	4	0.044	0.000	10.670
sko81	90998	2	0.018	0.010	25.020
sko90	115534	1	0.014	0.006	57.700
sko100a	152002	3	0.036	0.000	99.920
sko100b	153890	3	0.014	0.000	107.780
sko100c	147862	3	0.004	0.000	168.410
sko100d	149576	0	0.071	0.056	135.680
sko100e	149150	5	0.001	0.000	150.610
sko100f	149036	0	0.042	0.033	164.280
scr12	31410	10	0.000	0.000	0.011
scr15	51140	10	0.000	0.000	0.015
scr20	110030	10	0.000	0.000	0.026
tho30	149936	10	0.000	0.000	0.097
tho40	240516	6	0.012	0.000	2.956
tho150	8133398	0	0.055	0.036	202.586
wil50	48816	10	0.000	0.000	2.863
wil100	273038	0	0.010	0.005	78.965

表格 3. MTS 於題組三之實驗結果(共 53 題)

Type 3 - Real-Life Instances					
Problem	BKS	Times	APD	BPD	Time(min.)
bur26a	5426670	10	0.000	0.000	0.017
bur26b	3817852	10	0.000	0.000	0.023
bur26c	5426795	10	0.000	0.000	0.021
bur26d	3821225	10	0.000	0.000	0.022
bur26e	5386879	10	0.000	0.000	0.019
bur26f	3782044	10	0.000	0.000	0.025
bur26g	10117172	10	0.000	0.000	0.021
bur26h	7098658	10	0.000	0.000	0.026
els19	17212548	10	0.000	0.000	0.011
had12	1652	10	0.000	0.000	0.008
had14	2724	10	0.000	0.000	0.009
had16	3720	10	0.000	0.000	0.011
had18	5358	10	0.000	0.000	0.014
had20	6922	10	0.000	0.000	0.016
chr12a	9552	10	0.000	0.000	0.007
chr12b	9742	10	0.000	0.000	0.006
chr12c	11156	10	0.000	0.000	0.007
chr15a	9896	10	0.000	0.000	0.011
chr15b	7990	10	0.000	0.000	0.011
chr15c	9504	10	0.000	0.000	0.012
chr18a	11098	10	0.000	0.000	0.022
chr18b	1534	10	0.000	0.000	0.024
chr20a	2192	10	0.000	0.000	0.025
chr20b	2298	10	0.000	0.000	0.025
chr20c	14142	10	0.000	0.000	0.024
chr22a	6156	10	0.000	0.000	0.029
chr22b	6194	10	0.000	0.000	0.042
chr25a	3796	10	0.000	0.000	0.044
kra30a	88900	10	0.000	0.000	0.018
kra30b	91420	10	0.000	0.000	0.032
kra32	88700	10	0.000	0.000	0.020
ste36a	9526	10	0.000	0.000	0.201
ste36b	15852	10	0.000	0.000	0.692
ste36c	8239110	10	0.000	0.000	0.344
esc16a	68	10	0.000	0.000	0.000
esc16b	292	10	0.000	0.000	0.000
esc16c	160	10	0.000	0.000	0.000
esc16d	16	10	0.000	0.000	0.000
esc16e	28	10	0.000	0.000	0.000
esc16f	0	10	0.000	0.000	0.000
esc16g	26	10	0.000	0.000	0.000
esc16h	996	10	0.000	0.000	0.000
esc16i	14	10	0.000	0.000	0.000
esc16j	8	10	0.000	0.000	0.000
esc32a	130	10	0.000	0.000	0.001
esc32b	168	10	0.000	0.000	0.001
esc32c	642	10	0.000	0.000	0.001
esc32d	200	10	0.000	0.000	0.001
esc32e	2	10	0.000	0.000	0.001
esc32g	6	10	0.000	0.000	0.001
esc32h	438	10	0.000	0.000	0.001

esc64a	116	10	0.000	0.000	0.126
esc128	64	10	0.000	0.000	0.897

表格 4. MTS 於題組四之實驗結果(共 9 題)

Type 4 - Real-Life Like Instances					
Problem	BKS	Times	APD	BPD	Time(min.)
tai20b	122455319	10	0.000	0.000	0.012
tai25b	344355646	10	0.000	0.000	0.031
tai30b	637117113	10	0.000	0.000	0.044
tai35b	283315445	10	0.000	0.000	0.254
tai40b	637250948	10	0.000	0.000	0.414
tai50b	458821517	10	0.000	0.000	5.097
tai60b	608215054	10	0.000	0.000	13.584
tai80b	818415043	10	0.000	0.000	187.817
tai100b	1185996137	0	0.020	0.010	151.204

4.3 實驗結果分析

表格 5 統計了 DivTS、ANGEL 和 MTS 演算法在各類題組的實驗數據。由表格 5 可以清楚的看出 DivTS 在題組一和題組二表現最佳；ANGEL 在題組四表現最佳；而我們提出的 MTS 則在題組三表現最佳。MTS 在題組一則表現較不好，主要原因是 tai 題組普遍解的較不好，如果我們花多一點時間，增加初始族群的大小，則解的品質一定會再提升，修改 Local Search 或是針對問題特性設計新的 Local Search 也是個不錯的方法。然而如果用 CPU 的時間來比較演算法的效能就不見得公平，因為每個演算法用的程式語言不同，CPU 架構不同，作業系統不同，記憶體大小也不同，因此 CPU time 就比較不具參考價值。

五. 結論

多重軌跡搜尋(Multiple Trajectory Search, MTS)演算法能在解空間中廣泛且有系統的搜尋，也就是廣度搜尋能力佳，也因為如此，MTS 比起一些超啟發演算法來的有效率，如果再搭配好的區域搜尋，一定可以找到品質好的解。然而本論文所提出的方法，大部分的時間都花在 Local Search 上。Local Search 1 有加一些 Heuristic 的概念，而 Local Search 2 則是藉由交換所有可能的 facility - pair，交換數共有 C_n^2 種可能(n 為問題的大小)，因此 Local Search 2 會花比較多的計算時間，雖然我們有提出計算成本的快速算法，但是如果頻繁的使用 Local Search 2 還是會花太多的時間，所以 Local Search 2 只能在特定的情況下使用。在未來的研究中如果能針對問題的特性，例如：統計流量矩陣或距離矩陣的一些資訊來選擇適當的

Local Search 或是直接調整 Local Search 的相關參數，勢必可以讓搜尋的效果更上一層樓而且可以讓計算時間用在該用的地方，讓 MTS 演算法在運作上更有效率。

參考文獻

- [1] Angel, E. and V. Zissimopoulos, *On the landscape ruggedness of the quadratic assignment problem*. Theoretical Computer Science, **263**(1-2), 2001, pp. 159-172.
- [2] Bousonocalzon, C. and M.R.W. Manning, *The Hopfield Neural-Network Applied to the Quadratic Assignment Problem*. Neural Computing & Applications, **3**(2), 1995, pp. 64-72.
- [3] Burkard, R.E., S.E. Karisch, and F. Rendl, *QAPLIB - A quadratic assignment problem library*. Journal of Global Optimization, **10**(4), 1997, pp. 391-403.
- [4] Connolly, D.T., *An Improved Annealing Scheme for the Qap*. European Journal of Operational Research, **46**(1), 1990, pp. 93-100.
- [5] Demirel, N.C. and M.D. Toksari, *Optimization of the quadratic assignment problem using an ant colony algorithm*. Applied Mathematics and Computation, **183**(1), 2006, pp. 427-435.
- [6] Drezner, Z., *A new genetic algorithm for the quadratic assignment problem*. Informs Journal on Computing, **15**(3), 2003, pp. 320-330.
- [7] Drezner, Z., *Compounded genetic algorithms for the quadratic assignment problem*. Operations Research Letters, **33**(5), 2004, pp. 475-480.
- [8] Drezner, Z., *Extensive experiments with hybrid genetic algorithms for the solution of the quadratic assignment problem*. Computers & Operations Research, **35**(3), 2008, pp. 717-736.
- [9] Fleurent, C. and F. Glover, *Improved constructive multistart strategies for the quadratic assignment problem using adaptive memory*. Informs Journal on Computing, **11**(2), 1999, pp. 198-204.
- [10] Gambardella, L.M., E.D. Taillard, and M. Dorigo, *Ant colonies for the quadratic assignment problem*. Journal of the Operational Research Society, **50**(2), 1999, pp. 167-176.
- [11] James, T., C. Rego, and F. Glover, *A cooperative parallel tabu search algorithm for the quadratic assignment problem*. European Journal of Operational Research, **195**(3), 2009, pp. 810-826.
- [12] James, T., C. Rego, and F. Glover, *Multistart Tabu Search and Diversification Strategies for the Quadratic Assignment Problem*. IEEE Transactions on Systems Man and Cybernetics Part a-Systems and Humans, **39**(3), 2009, pp.

- 579-596.
- [13] Maniezzo, V. and A. Colomi, *The ant system applied to the quadratic assignment problem*. Ieee Transactions on Knowledge and Data Engineering, **11**(5), 1999, pp. 769-778.
- [14] Misevicius, A., *Genetic algorithm hybridized with ruin and recreate procedure: application to the quadratic assignment problem*. Knowledge-Based Systems, **16**(5-6), 2003, pp. 261-268.
- [15] Misevicius, A., *A tabu search algorithm for the quadratic assignment problem*. Computational Optimization and Applications, **30**(1), 2005, pp. 95-111.
- [16] Nissen, V. and H. Paul, *A Modification of Threshold Accepting and Its Application to the Quadratic Assignment Problem*. Or Spektrum, **17**(2-3), 1995, pp. 205-210.
- [17] Sahni, S. and T. Gonzalez, *P-Complete Approximation Problems*. Journal of the ACM, **23**(3), 1976, pp. 555-565.
- [18] Stutzle, T., *Iterated local search for the quadratic assignment problem*. European Journal of Operational Research, **174**(3), 2006, pp. 1519-1539.
- [19] Taillard, E., *Robust Taboo Search for the Quadratic Assignment Problem*. Parallel Computing, **17**(4-5), 1991, pp. 443-455.
- [20] Tate, D.M. and A.E. Smith, *A Genetic Approach to the Quadratic Assignment Problem*. Computers & Operations Research, **22**(1), 1995, pp. 73-83.
- [21] Tseng, L.Y. and S.C. Liang, *A hybrid metaheuristic for the quadratic assignment problem*. Computational Optimization and Applications, **34**(1), 2006, pp. 85-113.
- [22] Wilhelm, M.R. and T.L. Ward, *Solving Quadratic Assignment Problems by Simulated Annealing*. IIE Transactions, **19**(1), 1987, pp. 107-119.

表格 5. MTS 與其他兩種方法之比較

比較	DivTS			ANGEL			MTS		
	APD	BPD	Time	APD	BPD	Time	APD	BPD	Time
題組一	0.121	0.082	20.17	0.220	0.076	NA	0.190	0.082	19.87
題組二	0.005	0.001	43.12	0.010	0.003	NA	0.010	0.004	33.96
題組三	0.024	0.000	4.05	0.020	0.000	NA	0.000	0.000	0.038
題組四	0.007	0.001	23.40	0.000	0.000	NA	0.002	0.001	39.83